COMP1511 26T3 · Assignment 1：CS Submarine 要求整理

编写日期：2026-10-15 · 依据官方 spec（Change Log 版本 1.1，2026-10-15 16:00 更新）整理的中文概括，**不是原文**。命令、函数名、输出字符串以官方 spec 为准，**提交前务必对照原文**：
https://cgi.cse.unsw.edu.au/~cs1511/26T3/assignments/ass1/index.html

关键信息
- 截止：**2026 年 10 月 26 日（W7 周一）18:00**（悉尼时间）。
- 迟交：每晚一天，可得的最高分降低 5%（从满分上限扣，例如晚 1 天最高 95%，晚 5 天最高 75%）；**超过 5 天不再接受**（有特殊安排者除外）。
- 提交命令：`give cs1511 ass1_cs_submarine cs_submarine.c`（可多次提交，只评最后一次）。
- 占课程总分 **20%**；个人作业，禁止合作、禁止使用 AI / Copilot 等工具生成代码。

1. 游戏概述

玩家输入命令驾驶潜艇在一片网格海域中行动：收集海底所有 Treasure，然后从 Exit 逃出，且不能耗尽氧气。程序分两个阶段：

- **Setup Phase**：初始化空海域，用命令放置各种 entity（Treasure、Air Pocket、Mine、Exit、Seaweed、Anglerfish、Jellyfish、Current、Seeking Mine），最后放入潜艇。
- **Gameplay Phase**：潜艇移动（左右边界相互连通，即 horizontal wrap）、收集宝藏、在 Air Pocket 补氧、发射 torpedo，同时躲避各种危险；宝藏清空后 Exit 解锁，进入 Exit 即获胜。

2. 起步

```
mkdir ass1
cd ass1
1511 fetch-activity cs_submarine     # 获取 starter code：cs_submarine.c
1511 autotest cs_submarine           # 一开始测试会全部失败，属正常
1511 cs_submarine                    # 运行官方 reference implementation，观察正确行为
```

- **Starter code**：提供 `struct tile board[ROWS][COLS]`（每个 tile 目前只有一个 `enum entity` 字段）、`initialise_board()`（全部设为 `OCEAN`）、`print_board(board, row, col)`（潜艇尚未放置时传 `NO_SUBMARINE`），以及以后要调用的 `print_submarine_stats()`、`print_map_statistics()`（需传入 7 个计数）、`print_win_message()`、`print_oxygen_loss_message()`、`print_loss_message(entity)`。另有几个由 `print_board` 内部使用的辅助函数，不需要你调用。
- Row 0 是海面，row `ROWS - 1` 是紧贴海底的一行。
- **潜艇不是 entity，不存入 board**，要自己记录它的行列；打印为 `[o>`，画在所在 tile 之上。
- FAQ：可以自由修改或替换提供的常量、enum、struct 和函数（后面的 stage 默认你会给 `struct tile` 加字段），并鼓励自己新增；可以使用指针，但完成全部 stage 并不需要指针；课程没教的 C 特性要查 Style Guide（标"avoid"的用不好可能扣分，标"illegal / do not use"的一定扣分）。

- 测试保证：不会输入不存在的命令、类型错误的参数或个数不对的参数；每条命令都以一个 char 开头。
- 工具：可以把 setup 命令写进 `.in` 文件，用 `cat my_level.in - | ./cs_submarine` 运行（完成 2.1 之后用带 `-` 的写法，这样文件读完后还能继续从键盘输入）。

3. Stage 1：Setup Phase（●○○）

子阶段	要求概括
1.1 添加简单 entity	打印 Setup Phase 标题和提示后，循环读入 `t / o / m / x [row] [col]`（Treasure、Air Pocket、Mine、locked Exit），直到 Ctrl+D；输入结束后打印一次棋盘（读入过程中不打印）。
1.2 校验放置	坐标不在棋盘内，或目标 tile 已被占用时，拒绝放置并打印对应错误信息，然后继续读命令；每条被拒命令只打印一条信息。
1.3 结束 setup 并放置潜艇	新命令 `e`：棋盘上至少有一个 locked Exit 才能结束 setup，否则打印提示并继续；结束后提示输入潜艇起点，读入行列，打印带潜艇的棋盘，再打印 Gameplay Phase 标题。**Ctrl+D 与 `e` 行为不同**：Ctrl+D 只打印棋盘就结束程序，这个行为要一直保持。
1.4 校验潜艇起点	起点不在棋盘内或该 tile 不是 `OCEAN` 时打印错误信息并重新提示，直到输入合法为止。
1.5 Seaweed	`s [row] [col] [direction] [length]`：从起点开始放一串 Seaweed，`h` 向右、`v` 向上（朝海面）。路径上只要有一格非法就整串都不放；同时存在"越界"和"被占用"两种错误时，只报越界错误；Seaweed 不会左右环绕。

4. Stage 2：Gameplay Phase——潜艇回合（●○○）

子阶段	要求概括
2.1 移动	Gameplay loop 处理 `q`（打印退出信息后立即结束；Ctrl+D 结束时不打印这条信息）、`w/a/s/d`（上/左/下/右各移动一格）、`r`（原地休息，也算一个回合）。左右方向环绕，上下不环绕。每回合结束打印棋盘。氧气初始 100，每回合（包括休息）消耗 5，此时就要开始记录。
2.2 校验移动	目标是 Seaweed 或 locked Exit、在海面还往上、在海底还往下，都属于非法移动：打印提示，潜艇不动，**不算回合**（不耗氧、不打印棋盘）。注意环绕后的目标格也要检查。
2.3 收集与 `p`	移动后检查所在格：Treasure 计数 +1，该格改为 `OCEAN`；Air Pocket 氧气 +20，上限 100，该格改为 `OCEAN`。**先扣氧再收集**。`p` 调用 `print_submarine_stats()`，显示氧气、宝藏数、回合数、剩余 torpedo（本 stage 恒为 4）；`p` 不算回合，也不打印棋盘。
2.4 `m` 地图统计	按顺序数出当前棋盘上 Treasure、Air Pocket、Seaweed、Mine、Jellyfish、Anglerfish 的数量，以及 Exit 总数（含锁定和已解锁），传给 `print_map_statistics()`；`m` 不算回合。
2.5 胜负	棋盘上没有 Treasure 时，所有 Exit 一起解锁。停在已解锁 Exit 上即获胜；氧气降到 0 判负；停在 Mine 上判负（传 `MINE`）。游戏结束时依次输出：最后一次棋盘 → 胜/负信息 → 潜艇信息，然后退出。

回合顺序（Stage 2）：移动/休息 → 扣氧 → 收集（若宝藏清空则解锁 Exit）→ 检查是否获胜 → 检查氧气 → 检查 Mine → 打印棋盘。所以同一回合里，到达 Exit 的判定先于氧气耗尽；在最后一口气时吃到 Air Pocket 也能活下来。

5. Stage 3：海洋反击（●●◌）

子阶段	要求概括
3.1 Anglerfish	Setup 命令 `a [row] [col]`。未被发现时显示为普通海水；潜艇进入其 2 行且 2 列范围内（列距离按环绕计算）就会被发现，之后一直保持可见。每次打印棋盘时都要按当时潜艇位置判断。停在 Anglerfish 上判负（即使从未见过它）。统计数包括未发现的 Anglerfish，它所在的格也算被占用。
3.2 Deep Sea	`v` 切换深海模式（打印开启/关闭提示后再打印棋盘；`v` 不算回合）。开启时：潜艇在第 n 行时，可视半径为 `ROWS - n`（按欧几里得距离计算，列距离取环绕后较短的那个），半径外的格显示为 `###`；每回合耗氧改为"所在行号 + 1"。氧气可以变成负数；Air Pocket 在判负之前收集。看不见的东西照样生效。
3.3 Jellyfish	Setup 命令 `j [axis] [row] [col]`：`v` 为竖直巡逻（先向上），`h` 为水平巡逻（先向右）。每回合在潜艇行动结算之后，所有 Jellyfish **同时**按回合开始时的棋盘移动：前方格不是 `OCEAN`（竖直方向的海面/海底也算挡住）就原地掉头，否则前进；两只要进入同一格时都掉头。潜艇不算阻挡物。Jellyfish 移到潜艇所在格则判负。非法移动和 `v` 都不会让 Jellyfish 移动。
3.4 Undo	`u [turns]`：回退若干回合，恢复整个棋盘、潜艇位置/氧气/宝藏数/回合数、每只 Jellyfish 的位置和方向、Anglerfish 是否已被发现；最多退回到 Gameplay 开始时；完成后打印棋盘。Undo 不算回合，也不改变 Deep Sea 模式。保证总回合数不超过 256。
3.5 Ocean Currents	Setup 命令 `c [w/a/s/d] [row] [col]` 放置四个方向的洋流。每回合潜艇若站在洋流上，就被推一格，可连续推，最多 5 次；被 Seaweed、locked Exit 或上下边界挡住时停止。被推到危险物上立即判负。在本回合中游上洋流的 Jellyfish 也会被推动（按从上到下、从左到右的顺序逐只处理），但保持自己的巡逻方向。所有洋流每 3 回合顺时针旋转 90°，作为回合的最后一步。潜艇也不能以洋流格为起点。Undo 还要恢复洋流方向。

回合顺序（至 3.5）：移动/休息 → 扣氧 → 洋流推动潜艇 → 收集/解锁 → 检查获胜 → 检查氧气 → 检查所在格的危险物 → Jellyfish 移动并被洋流推动 → 检查 Jellyfish 是否撞上潜艇 → 每 3 回合旋转洋流 → 打印棋盘。

6. Stage 4：额外机制（●●●）

官方说明：这一阶段需要大量规划、重构和手动测试，tutor 不会就本阶段的调试或代码给出具体建议。

子阶段	要求概括
4.1 Torpedoes	`f [w/a/s/d]`：整局共 4 枚。发射算一个回合（耗氧，潜艇不动，但脚下的洋流照样推它）；没有 torpedo 时打印提示，不算回合。每回合先让已在飞行中的 torpedo 各前进 1 格，再让本回合发射的 torpedo 从潜艇最终位置出发前进 2 格。torpedo 进入一格时按规则结算：穿过海水和洋流；替潜艇收集 Treasure 或 Air Pocket；摧毁 Jellyfish、Anglerfish、Mine 后自身消失；碰到 Seaweed 或 Exit 被吸收；两枚 torpedo 相遇同归于尽；碰到潜艇判负。左右会环绕，越过上下边界后消失。Undo 还要恢复 torpedo 剩余数和所有飞行中的 torpedo。
4.2 Seeking Mines	`S [row] [col] [speed] [script]X`：script 由 w/a/s/d 组成，以大写 `X` 结尾（需逐字符读入）。每回合按 script 走 `speed` 步，走完一遍后从头循环；多颗 mine 按放置顺序一轮一步地轮流行动。Seaweed、Exit 和上下边界会挡住这一步（挡住了也算用掉一步）；会摧毁所经过的 Treasure、Air Pocket、Mine、Jellyfish、Anglerfish、torpedo（不算收集，但宝藏被清空时同样解锁 Exit）；多颗 Seeking Mine 之间互相穿过；任何一步进入潜艇所在格都立即判负。只有 torpedo 能摧毁它。统计时计入 Mine 数量。行动时机在 torpedo 之后。Undo 还要恢复每颗 mine 的位置和 script 进度。

Extension（SplashKit 图形界面）：不计分、没有 autotest；可以在论坛的 SplashKit Gallery 分享截图或视频，但**不得分享代码**。

7. 测试与提交流程

```
1511 autotest-stage 01_01 cs_submarine   # 测试某个子阶段（01_01 … 04_02）
1511 autotest-stage 01 cs_submarine      # 测试整个 stage
1511 style cs_submarine.c                # 风格检查
give cs1511 ass1_cs_submarine cs_submarine.c
```

- 官方建议"give early, give often"：中间版本无法编译也可以提交，只评最后一次提交。每次 autotest 或 give 都会保存一份备份。
- autotest 不能代替自己的测试；每个子阶段都要和 reference implementation 对比输出。

8. 评分方案

部分	占比	说明
Performance（自动测试）	80%	大致为：Stage 1 全部正确 → 35%；Stage 1–2 → 65%；Stage 1–3 → 85%；Stage 1–4 → 100%
Style（人工批改）	20%	可读性与清晰度，包括函数的使用，以及循环和 if 用得是否高效（见下表）

8.1 Style Marking Rubric（满分 20，概括）

类别	项目（分值）	满分要求
Formatting（5）	Indentation（2）	全程缩进一致
	Whitespace（1）	运算符两侧等空格使用一致
	Vertical whitespace（1）	代码段之间合理空行
	Line length（1）	没有超过 80 字符的行
Documentation（5）	Comments（3）	各代码段和函数上方有说明用途的注释；有包含姓名、zID 和程序说明的 header comment
	Naming（2）	函数、变量、常量名都符合 Style Guide 并且有助于理解
Organisation（5）	Function usage（4）	全部代码合理拆分成函数，**每个函数（包括 main）不超过 50 行**
	Function prototypes（1）	所有函数都在 main 上方有 prototype
Elegance（5）	Overdeep nesting（2）	没有 5 层及以上的嵌套
	Code repetition（2）	可能重复的代码已用函数或循环处理
	Constant usage（1）	所有 magic number 都用 `#define`，且使用一致
Illegal elements	出现 Style Guide 中列为 illegal 的元素：style 分**最高只能得 16/20**	

允许使用的 C 特性：除 Style Guide 的限制外没有其它限制；但如果使用课程没教的特性，可能得不到课程人员的帮助。

9. 学术诚信与作业规则（概括）

- 个人作业，提交内容必须完全是自己写的；不得提交任何部分由他人编写的代码（spec 页面上关于"少量代码"例外的那句话在页面上显示不完整，具体条件「未发布」，以官方页面或询问课程为准）。
- 提交会被自动和人工检查是否含他人代码。只能向 COMP1511 教学人员求助（论坛、help session 等）；不要在论坛贴作业代码。
- 禁止使用 GitHub Copilot 等代码生成工具或生成式 AI 来生成代码。
- 课程期间和课程结束后，都不得分享、发布或展示自己的作业代码（例如放到公开 GitHub 仓库）。
- 违规可能引发学术诚信调查，后果最重可至 COMP1511 记 0 分并被 UNSW 开除。按 spec 的罚则：故意把作业提供给他人且被提交、或提交他人的作业（包括合作完成的），该作业 0 分；付钱请人代写、或未经他人同意提交其作业，课程记 0 FL。

10. 依赖的每周知识点

周次	Ass1 用到的内容
W1	`printf` / `scanf`、format specifier、`" %c"` 读命令字符、`#define` 常量、dcc 编译
W2	if / else-if 链处理不同命令；`while (scanf(...) == n)` 循环读到 Ctrl+D；嵌套循环；`struct`（tile）与 `enum`（entity、方向）
W3	拆分函数与 prototype；pass by value（需要用返回值或其它方式更新状态）；Style Guide 与 `1511 style`；for 循环；数组（如保存 torpedo、Seeking Mine 及其 script、undo 历史）
W4	**2D array**（board）的遍历、计数和边界检查；array of structs；2D 数组作为函数参数；按字符读入 script
W5	讲座的 Mud and Bones（2D array of structs + enums 的移动游戏）几乎就是 Ass1 的缩小版：坐标校验、wasd 移动、收集与计数、胜负判定；Ass1 style 提示；指针（可选，FAQ 说明不是必须）

另：Stage 3.2 需要计算欧几里得距离（spec 给出了公式）；Stage 3.4 的 undo 需要保存历史状态。这两点 W1–W5 的讲座中没有专门讲授对应写法，需要自行设计。

来源页面

- Assignment 1 官方 spec：https://cgi.cse.unsw.edu.au/~cs1511/26T3/assignments/ass1/index.html
- Week 4 Lecture 1 幻灯片（Ass1 介绍）：https://cgi.cse.unsw.edu.au/~cs1511/26T3/slides/week_4/COMP1511_26T3_Week_4_Lecture_1_Tuesday.pdf
- Week 5 Lecture 1 幻灯片（Mud and Bones、Ass1 style 提示）：https://cgi.cse.unsw.edu.au/~cs1511/26T3/slides/week_5/COMP1511_26T3_Week_5_Lecture_1.pdf
- Style Guide：https://cgi.cse.unsw.edu.au/~cs1511/26T3/resources/style_guide.html

说明：本文件为概括整理，省略了所有示例输入输出和精确的输出字符串；实现时请以官方 spec 原文和 reference implementation（`1511 cs_submarine`）为准。